

IHK AP2 – Testtheorie & Kurzantworten 2025 (FIAE)

Kompaktes Handbuch für den Theorie-Teil: Whitebox/Blackbox, Regression, Abdeckungen (Anweisung, Zweig, Pfad), Testfalldesign, Rekursion vs. Iteration, Musterantworten im IHK-Stil.

1) Whitebox vs. Blackbox

Whitebox-Test – Vorteile

- **Fehler lokalisierbar:** genaue Stelle/Zweig im Code ermittelbar.
- **Hohe Testabdeckung messbar:** Pfade/Zweige/Anweisungen gezielt abdecken.
- **Dead Code/Komplexität sichtbar:** Optimierung des Quellcodes möglich.
- **Frühe Tests:** bereits während der Implementierung.

Blackbox-Test – Vorteile

- **Spezifikationsprüfung:** testet, was *gefordert* ist, unabhängig vom Code.
- **Benutzersicht:** realistische Eingaben/Anwendungsfälle.
- **Technologieunabhängig:** auch bei externen Systemen/Modulen.

Formulierungshilfe (IHK):

„Whitebox-Tests kennen den Quellcode. Vorteile: (1) Fehler lassen sich präzise lokalisieren. (2) Hohe Abdeckung von Anweisungen und Zweigen ist gezielt erreichbar. (3) Der Code kann dadurch optimiert werden.“

2) Regressionstest – Zweck & Einsatz

Definition: Regressionstests stellen sicher, dass vorhandene Funktionalität nach Änderungen (neuer Code, Refactoring, Bugfix, Performance-Optimierung) weiterhin korrekt funktioniert.

- **Ziel:** Wiederkehr von alten Fehlern verhindern (keine „Regression“).
- **Methodik:** Vorhandene Testfälle erneut ausführen, ggf. um neue Fälle erweitern.
- **Zeitpunkt:** bei jedem Build, nach jedem Merge/Release.

Formulierungshilfe (IHK):

„Regressionstests prüfen nach Änderungen, dass bestehende Funktionen nicht beeinträchtigt sind und weiterhin den Anforderungen entsprechen.“

3) Abdeckungsgrade: Anweisung, Zweig, Pfad

3.1 Anweisungsüberdeckung (Statement Coverage)

Prozentanteil der *ausgeführten Anweisungen*. Ziel: Jede Anweisung wird mindestens einmal ausgeführt.

Beispiel:

```
if (x > 0) a = 1; else a = 2; b = 3;
```

Ein Test mit $x=5$ deckt $a=1$ und $b=3$ ab, aber nicht den `else`-Teil → Anweisungsüberdeckung < 100%.

Antwortstil: „Maß, wie viele Anweisungen von Testfällen ausgeführt werden; Ziel: alle mindestens einmal.“

3.2 Zweigüberdeckung (Branch Coverage)

Prozentanteil der *verwendeten Zweige* (true/false-Wege jeder Bedingung). Ziel: Jede Bedingung einmal true und einmal false.

Beispiel: Für `if (x>0)` braucht man mindestens zwei Tests: $x=5$ (true) und $x=-1$ (false).

Merke: Volle Zweigüberdeckung impliziert immer volle Anweisungsüberdeckung.

3.3 Pfadüberdeckung (Path Coverage)

Prozentanteil der *verschiedenen vollständigen Pfade* vom Eintritt bis zum Austritt. Sehr aufwändig/oft unmöglich bei Schleifen (theoretisch unendlich).

Ein *Pfad* ist eine Folge von Anweisungen/Verzweigungen von Start bis Ende (z. B. $B1 \rightarrow A1 \rightarrow B2 \rightarrow A3 \rightarrow B3 \rightarrow A4$).

Merke: Volle Pfadüberdeckung $\Rightarrow$ volle Zweig- und Anweisungsüberdeckung.

4) „Wie viele Testfälle sind nötig?“ – Vorgehen

1. **Kontrollfluss skizzieren:** Blöcke (A1,A2,...) und Bedingungen (B1,B2,...) markieren.
2. **Für Anweisungsüberdeckung:** Wähle minimal so, dass jeder Block mindestens einmal auf einem Pfad liegt.
3. **Für Zweigüberdeckung:** Sorge bei jeder Bedingung für true *und* false (mind. zwei Tests pro Bedingung – oft kombinierbar).
4. **Für Pfadüberdeckung:** Alle relevanten Pfade einmal abdecken (bei Schleifen meist auf 0/1-Durchlauf begrenzen).

Mini-Beispiel: Wenn ein Fluss B1 (T/F) und danach B2 (T/F) hat, aber bei B1=false wird B2 übersprungen: Für *Anweisungsüberdeckung* reichen meist 2 Tests (z. B. B1=T über B2, B1=F). Für *Zweigüberdeckung* brauchst du zusätzlich B2=F, also mind. 3 Tests – sofern T/F-Kombinationen nicht in einem Fall gleichzeitig abgedeckt werden können.

Antwortschablone: „Für Anweisungsüberdeckung sind n Testfälle erforderlich, da ... (jeder Block A1..Ax wird mindestens einmal erreicht).“

5) „Beschreiben Sie den Fehler und eine Korrekturmöglichkeit“

Vorgehensweise:

1. Fehlerart benennen (Logik, Grenze, Initialisierung, falscher Operator, fehlender else-Zweig, falscher Rückgabewert).
2. Minimalbeispiel nennen, das den Fehler zeigt (z. B. Eingabe 0 statt >0).
3. Konkrete Korrektur angeben (z. B. $> \rightarrow \geq$, Zähler vor Schleife initialisieren, Rückgabe ans Ende).

Beispielantwort:

„Fehler: Rückgabe steht in der Schleife $\rightarrow$ bricht zu früh ab. Korrektur: `return` hinter die Schleife verschieben (nur bei Treffer in der Suche innerhalb der Schleife).“

6) Unittest – Definition & Zweck

Definition: Test einer kleinsten unabhängigen Einheit (Funktion, Methode, Klasse) isoliert vom Rest.

- **Zweck:** früh Fehler finden, Verhalten spezifizieren, Änderungen absichern (Regression), Dokumentation.
- **AAA-Muster:** Arrange (Daten anlegen), Act (Methode aufrufen), Assert (Ergebnis prüfen).

Beispielsatz (IHK): „Ein Unittest prüft eine einzelne Methode isoliert. Mit Arrange-Act-Assert wird erwartetes Verhalten automatisch verifiziert.“

7) Testfalldesign: Äquivalenzklassen & Grenzwertanalyse

Äquivalenzklassen

Eingabebereich in gültige/ungültige Klassen zerlegen; pro Klasse 1 repräsentativer Test.

z. B. Alter 0–120: Klassen <0 (ungültig), $0\text{--}120$ (gültig), >120 (ungültig).

Grenzwertanalyse

Werte direkt an Grenzen testen (min , $\text{min}\pm 1$, max , $\text{max}\pm 1$).

z. B. Rabatt ab 100€: teste 99, 100, 101.

Formulierhilfe: „Wir definieren Äquivalenzklassen und testen die Grenzwerte, um mit wenigen Fällen eine hohe Fehlerentdeckungsrate zu erzielen.“

8) Rekursion vs. Iteration – je ein Vor- und Nachteil

Vorteil Rekursion

- Oft **einfacher/übersichtlicher** bei Baum/Teil-Problemen (z. B. Quicksort, Baumtraversierung).
- Natürliche Problembeschreibung (Teile-und-Herrsche).

Nachteil Rekursion

- **Stack-Overflow-Risiko** bei tiefer Rekursion.
- Mehr Overhead (Funktionsaufrufe), oft schlechtere Laufzeit als Iteration.

Beispielantwort (IHK):

„Vorteil: rekursive Lösungen sind kompakt und folgen der Problemlogik (z. B. Baum). Nachteil: Gefahr von Stack-Überlauf und zusätzlicher Aufruf-Overhead; iterativ oft effizienter.“

Last-Minute-Checkliste (Theorie)

- Whitebox-Vorteile: Fehler lokalisieren, hohe Abdeckung, Optimierung.
- Regression: nach Änderung prüfen, dass Altes noch funktioniert.
- Abdeckung: Anweisung < Zweig < Pfad (Implikationskette).
- AAA-Muster bei Unit-Tests.
- Grenzwerte + Äquivalenzklassen nennen.
- Rekursion: +einfach / –Stack, –Overhead.